

TASK 3

SQL FOR DATA ANALYSIS

Ecommerce Dataset | SQLite Implementation

DOCUMENT FOLLOWS

A formal technical report covering database design, SQL implementation, analytical queries, results, optimization, and interview preparation.

Organization	Elevates Lab
Program	Data Analyst Internship
Task	Task 3: SQL for Data Analysis
Prepared By	Ganesh M

Submission-ready technical documentation

Executive Summary

This report documents the completion of Task 3: SQL for Data Analysis from the supplied Data Analyst Internship task sheet. The task requires an Ecommerce dataset and SQL analysis using SQLite, MySQL, or PostgreSQL. This implementation uses SQLite and demonstrates the required SQL operations through a structured Ecommerce database.

The project covers data retrieval, row filtering, sorting, grouped analysis, joins, aggregate functions, subqueries, NULL handling, analytical views, and indexes. The report also documents the interview questions specified in the task sheet and provides practical explanations of each concept.

DELIVERABLES

The task sheet specifies an SQL file containing queries and screenshots of outputs. The supporting GitHub submission should also contain the dataset, screenshots, and README.md.

Contents

1. Task Requirements
 2. Objective and Scope
 3. Tools and Technology
 4. Dataset and Database Design
 5. Data Model and Relationships
 6. SQL Implementation
 7. Data Analysis Queries and Results
 8. Advanced SQL Analysis
 9. Query Optimization
 10. Key Findings
 11. Interview Questions and Answers
 12. Submission Package
 13. Conclusion
- Appendix A: Complete SQL Script

1. Task Requirements

The supplied task sheet defines the objective as using SQL queries to extract and analyze data from a database. It specifies Ecommerce as the suggested dataset and lists MySQL, PostgreSQL, or SQLite as acceptable tools.

Requirement	Implementation in this report
SELECT, WHERE, ORDER BY, GROUP BY	Demonstrated with product and sales analysis
JOINS	INNER JOIN, LEFT JOIN, and multi-table joins
Subqueries	Orders above average and customer-level comparison
Aggregate functions	SUM, AVG, COUNT, MIN, MAX
Views	Customer revenue analytical view
Indexes	Customer, date, and category indexes
Deliverables	SQL script and output screenshots documented

The task sheet also lists interview questions on WHERE versus HAVING, join types, average revenue per user, subqueries, SQL optimization, views, and NULL handling. These are addressed later in this report.

2. Objective and Scope

Primary objective:

- Use SQL to manipulate, query, and analyze structured Ecommerce data.

Scope of analysis:

- Create a relational Ecommerce database.
- Load customers, products, orders, and order-item records.
- Perform basic retrieval and filtering.
- Combine related tables using joins.
- Calculate revenue and customer metrics.
- Use subqueries for comparative analysis.
- Create a reusable analytical view.
- Create indexes and inspect query execution planning.
- Prepare evidence through output screenshots.

3. Tools and Technology

Tool / Technology	Purpose
SQLite	Relational database engine used for implementation
SQL	Database definition, data manipulation, and analysis
Microsoft Word	Formal technical report
Python	Generation of supporting output images for documentation
GitHub	Required repository-based submission package

4. Dataset and Database Design

The Ecommerce database contains four related tables. The design separates customer information, product information, order-level information, and item-level information. This separation avoids storing repeated product and customer details inside every order row.

Table	Primary Key	Important Columns	Records
customers	customer_id	customer_name, email, city, country	8
products	product_id	product_name, category, price	8
orders	order_id	customer_id, order_date, total_amount	10
order_items	order_item_id	order_id, product_id, quantity, unit_price	18

4.1 Why Four Tables?

- Customers holds one record per customer.
- Products holds one record per product.
- Orders records each purchase transaction.
- Order_items records the products and quantities included in each order.
- Foreign keys connect the tables so an analyst can trace a purchase from customer to order to product.

4.2 Relationship Structure

```
customers.customer_id → orders.customer_id
orders.order_id       → order_items.order_id
products.product_id   → order_items.product_id
```

RELATIONAL FLOW

Customer → Order → Order Item → Product. This structure supports customer, transaction, and product-level analysis.

5. SQL Implementation

5.1 Table Creation

```
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY,
  customer_name TEXT NOT NULL,
  email TEXT,
  city TEXT,
  country TEXT
);

CREATE TABLE products (
  product_id INTEGER PRIMARY KEY,
  product_name TEXT NOT NULL,
  category TEXT,
  price REAL
);

CREATE TABLE orders (
  order_id INTEGER PRIMARY KEY,
  customer_id INTEGER,
  order_date DATE,
```

```

    total_amount REAL,
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
    order_item_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    unit_price REAL,
    FOREIGN KEY (order_id)
        REFERENCES orders(order_id),
    FOREIGN KEY (product_id)
        REFERENCES products(product_id)
);

```

The primary keys provide unique identifiers. Foreign keys establish relationships between tables. This structure allows SQL JOIN operations to combine records from different business entities.

5.2 Data Validation

```

SELECT COUNT(*) AS customer_count FROM customers;
SELECT COUNT(*) AS product_count FROM products;
SELECT COUNT(*) AS order_count FROM orders;
SELECT COUNT(*) AS order_item_count FROM order_items;

```

Expected record counts: 8 customers, 8 products, 10 orders, and 18 order items.

6. Data Analysis Queries and Results

6.1 SELECT

SELECT retrieves data from a table. Selecting only required columns produces a focused analytical result.

```

SELECT product_name, category, price
FROM products;

```

6.2 WHERE

WHERE filters individual rows based on a condition.

```

SELECT product_name, category, price
FROM products
WHERE category = 'Electronics';

```

INTERPRETATION

The query returns only products whose category is Electronics.

6.3 ORDER BY

```

SELECT product_name, category, price
FROM products
ORDER BY price DESC;

```

DESC sorts from highest to lowest. ASC sorts from lowest to highest.

6.4 GROUP BY

```
SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category;
```

GROUP BY forms groups based on a column. COUNT then calculates the number of records inside each group. This is useful for category-level reporting.

6.5 Aggregate Analysis

```
SELECT SUM(total_amount) AS total_revenue
FROM orders;

SELECT ROUND(AVG(total_amount), 2) AS average_order_value
FROM orders;

SELECT MIN(total_amount) AS minimum_order_value,
       MAX(total_amount) AS maximum_order_value
FROM orders;
```

Metric	Result
Total Revenue	₹199,500.00
Average Order Value	₹19,950.00
Minimum Order Value	₹1,800.00
Maximum Order Value	₹57,500.00

Output Evidence: Revenue Metrics

Total Revenue

total_revenue
199500.0

Generated result for total revenue.

Output Evidence: Average Order Value

Average Order Value

average_order_value
19950.0

Generated result for average order value.

7. JOIN Analysis

7.1 INNER JOIN

INNER JOIN returns rows where the join condition matches in both tables. It is appropriate when the analysis requires records with a valid relationship in both datasets.

```
SELECT o.order_id,
       c.customer_name,
       c.city,
       o.order_date,
       o.total_amount
FROM orders o
INNER JOIN customers c
      ON o.customer_id = c.customer_id;
```

7.2 LEFT JOIN

```
SELECT c.customer_id,
       c.customer_name,
       o.order_id,
       o.total_amount
FROM customers c
LEFT JOIN orders o
      ON c.customer_id = o.customer_id;
```

LEFT JOIN keeps all rows from the left table. If no matching order exists, order fields appear as NULL. This pattern is useful for identifying customers with no purchases.

7.3 Multi-Table JOIN

```
SELECT c.customer_name,
       o.order_id,
       p.product_name,
       oi.quantity,
       oi.unit_price
FROM customers c
INNER JOIN orders o
      ON c.customer_id = o.customer_id
INNER JOIN order_items oi
      ON o.order_id = oi.order_id
INNER JOIN products p
      ON oi.product_id = p.product_id;
```

This query connects all four tables and creates a detailed transaction-level result. It shows which customer placed which order and which product was included.

8. Product and Customer Analysis

8.1 Product Revenue

```
SELECT p.product_name,
       SUM(oi.quantity * oi.unit_price) AS product_revenue
FROM products p
INNER JOIN order_items oi
  ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY product_revenue DESC;
```

Product	Revenue
Laptop	₹110,000.00
Smartphone	₹50,000.00
Monitor	₹24,000.00
Headphones	₹5,000.00
Mouse	₹3,700.00
Backpack	₹3,600.00
Keyboard	₹3,000.00
USB Cable	₹500.00

Output Evidence: Product Revenue

Product Revenue

product_name	product_revenue
Laptop	110000.0
Smartphone	50000.0
Monitor	24000.0
Headphones	5000.0
Mouse	3700.0
Backpack	3600.0
Keyboard	3000.0
USB Cable	500.0

Product revenue ranking generated from order items.

8.2 Customer Revenue

```
SELECT c.customer_name,
       SUM(o.total_amount) AS total_revenue
FROM customers c
INNER JOIN orders o
      ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_revenue DESC;
```

Customer	Revenue
Ganesh	₹71,300.00
Kiran	₹55,000.00
Priya	₹28,300.00
Rahul	₹25,000.00
Arjun	₹12,500.00
Sneha	₹3,300.00
Meera	₹2,300.00
Anjali	₹1,800.00

Output Evidence: Customer Revenue

Customer Revenue

customer_name	total_revenue
Ganesh	71300.0
Kiran	55000.0
Priya	28300.0
Rahul	25000.0
Arjun	12500.0
Sneha	3300.0
Meera	2300.0
Anjali	1800.0

Customer revenue ranking generated from orders.

8.3 Revenue by City

```
SELECT c.city,
       SUM(o.total_amount) AS total_revenue
FROM customers c
INNER JOIN orders o
      ON c.customer_id = o.customer_id
GROUP BY c.city
ORDER BY total_revenue DESC;
```

City	Revenue
Mysuru	₹71,300.00
Mumbai	₹55,000.00
Chennai	₹28,300.00
Bengaluru	₹25,000.00
Delhi	₹12,500.00
Pune	₹3,300.00
Kochi	₹2,300.00
Hyderabad	₹1,800.00

9. Advanced SQL Analysis

9.1 Average Revenue Per User

<pre>SELECT ROUND(SUM(total_amount) * 1.0 / COUNT(DISTINCT customer_id), 2) AS average_revenue_per_user FROM orders;</pre>
--

Metric	Result
Average Revenue Per User	₹24,937.50

Output Evidence: Average Revenue Per User

Average Revenue Per User

average_revenue_per_user
24937.5

ARPU result.

Formula: ARPU = Total Revenue ÷ Number of Unique Customers.

9.2 Subquery

<pre>SELECT order_id, customer_id, order_date, total_amount FROM orders WHERE total_amount > (SELECT AVG(total_amount)</pre>
--

```
FROM orders
)
ORDER BY total_amount DESC;
```

Order	Customer ID	Date	Amount
1001	1	2026-09-01	₹57,500.00
1006	5	2026-09-08	₹55,000.00
1002	2	2026-09-02	₹25,000.00
1009	3	2026-09-15	₹25,000.00

Output Evidence: Subquery

Orders Above Average Order Value

order_id	customer_id	order_date	total_amount
1001	1	2026-09-01	57500.0
1006	5	2026-09-08	55000.0
1002	2	2026-09-02	25000.0
1009	3	2026-09-15	25000.0

Orders whose total is above the overall average.

9.3 WHERE versus HAVING

WHERE	HAVING
Filters individual rows.	Filters grouped results.
Used before GROUP BY.	Used after GROUP BY.
Example: total_amount > 5000.	Example: SUM(total_amount) > 30000.

```
-- WHERE
SELECT customer_id, SUM(total_amount) AS total_spent
FROM orders
WHERE total_amount > 5000
GROUP BY customer_id;

-- HAVING
SELECT customer_id, SUM(total_amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(total_amount) > 30000;
```

9.4 NULL Handling

```
SELECT customer_id, customer_name, email
FROM customers
```

```
WHERE email IS NULL;

SELECT customer_name,
       COALESCE(email, 'No Email') AS email
FROM customers;
```

IS NULL tests for missing values. COALESCE returns the first non-NULL expression. The current dataset contains populated customer email values, so the IS NULL query returns no records. The COALESCE query demonstrates the required handling method.

9.5 Analytical VIEW

```
CREATE VIEW customer_revenue AS
SELECT c.customer_id,
       c.customer_name,
       c.city,
       SUM(o.total_amount) AS total_revenue
FROM customers c
INNER JOIN orders o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name, c.city;

SELECT *
FROM customer_revenue
ORDER BY total_revenue DESC;
```

ID	Customer	City	Revenue
1	Ganesh	Mysuru	₹71,300.00
5	Kiran	Mumbai	₹55,000.00
3	Priya	Chennai	₹28,300.00
2	Rahul	Bengaluru	₹25,000.00
7	Arjun	Delhi	₹12,500.00
6	Sneha	Pune	₹3,300.00
8	Meera	Kochi	₹2,300.00
4	Anjali	Hyderabad	₹1,800.00

A view stores a reusable SQL definition. It is useful when the same business calculation needs to be accessed repeatedly.

10. Query Optimization

The task sheet asks for query optimization with indexes. Indexes were created on columns frequently used in joins and filtering.

```
CREATE INDEX idx_orders_customer_id
ON orders(customer_id);

CREATE INDEX idx_orders_order_date
ON orders(order_date);

CREATE INDEX idx_products_category
ON products(category);
```

The customer_id index supports customer-to-order joins and customer filters. The order_date index supports date-based filtering. The category index supports product category filtering.

```
EXPLAIN QUERY PLAN
SELECT *
FROM orders
WHERE customer_id = 1;
```

EXPLAIN QUERY PLAN provides SQLite's planned execution strategy. It helps identify whether the database is using a table scan or an available index.

11. Key Findings

- Total revenue recorded in the dataset is ₹199,500.00.
- Average order value is ₹19,950.00.
- Average revenue per user is ₹24,937.50.
- The highest-revenue customer in this dataset is Ganesh, with ₹71,300.00.
- The highest-revenue product is Laptop, with ₹110,000.00.
- The SQL workflow supports analysis at customer, order, city, category, and product levels.

12. Interview Questions and Answers

1. What is the difference between WHERE and HAVING?

WHERE filters individual rows before grouping. HAVING filters grouped results after GROUP BY. WHERE is used for row-level conditions, while HAVING is used for aggregate conditions.

2. What are the different types of joins?

Common join types are INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN. The task requires JOIN usage, and this implementation demonstrates INNER JOIN, LEFT JOIN, and multi-table joins.

3. How do you calculate average revenue per user in SQL?

Use SUM(total_amount) divided by COUNT(DISTINCT customer_id). DISTINCT ensures each customer is counted once.

4. What are subqueries?

A subquery is a query nested inside another SQL statement. The above-average order analysis uses a subquery to calculate the overall average order value.

5. How do you optimize a SQL query?

Use suitable indexes, retrieve only required columns, avoid unnecessary joins, apply appropriate filters, and inspect the execution plan.

6. What is a view in SQL?

A view is a reusable virtual table based on a SQL query. The customer_revenue view stores the customer revenue calculation for repeated use.

7. How would you handle NULL values in SQL?

Use IS NULL or IS NOT NULL to test missing values. Use COALESCE to provide a fallback value when a field is NULL.

13. Submission Package

The task sheet requires a GitHub repository for each task and asks for the code, dataset, screenshots, and a short README.md.

```
Task-3-SQL-Data-Analysis/
├── ecommerce_analysis.sql
├── ecommerce_database.db
├── README.md
├── dataset/
│   └── ecommerce_dataset.csv
├── screenshots/
│   ├── database_verification.png
│   ├── select_output.png
│   ├── where_output.png
│   ├── order_by_output.png
│   ├── group_by_output.png
│   ├── join_output.png
│   ├── revenue_output.png
│   ├── arpu_output.png
│   ├── subquery_output.png
│   ├── view_output.png
│   └── index_output.png
```

FINAL CHECK

Before submission, verify that the SQL file runs successfully, screenshots show the query and output clearly, README.md explains the work, and the GitHub repository contains the complete task package.

14. Conclusion

This project provides a complete SQL-based Ecommerce analysis workflow aligned with the supplied Task 3 requirements. The database design separates customers, products, orders, and order items. The analysis then builds from basic SELECT and WHERE operations to grouped metrics, joins, subqueries, views, NULL handling, and indexes.

The resulting report documents both the technical implementation and the analytical purpose of each SQL operation. It is structured as a formal internship submission and includes output evidence and interview preparation.

Appendix A: Complete SQL Script

The following consolidated script represents the database setup and core analysis used in this report.

```
PRAGMA foreign_keys = ON;

DROP VIEW IF EXISTS customer_revenue;
DROP TABLE IF EXISTS order_items;
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS customers;

CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT NOT NULL,
    email TEXT,
    city TEXT,
    country TEXT
);

CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT NOT NULL,
    category TEXT,
    price REAL
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER,
    order_date DATE,
    total_amount REAL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

CREATE TABLE order_items (
    order_item_id INTEGER PRIMARY KEY,
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    unit_price REAL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Insert data
INSERT INTO customers VALUES
(1, 'Ganesh', 'ganesh@gmail.com', 'Mysuru', 'India'),
(2, 'Rahul', 'rahul@gmail.com', 'Bengaluru', 'India'),
(3, 'Priya', 'priya@gmail.com', 'Chennai', 'India'),
(4, 'Anjali', 'anjali@gmail.com', 'Hyderabad', 'India'),
(5, 'Kiran', 'kiran@gmail.com', 'Mumbai', 'India'),
(6, 'Sneha', 'sneha@gmail.com', 'Pune', 'India'),
(7, 'Arjun', 'arjun@gmail.com', 'Delhi', 'India'),
(8, 'Meera', 'meera@gmail.com', 'Kochi', 'India');

INSERT INTO products VALUES
(101, 'Laptop', 'Electronics', 55000),
(102, 'Smartphone', 'Electronics', 25000),
(103, 'Headphones', 'Electronics', 2500),
(104, 'Keyboard', 'Accessories', 1500),
(105, 'Mouse', 'Accessories', 800),
(106, 'Monitor', 'Electronics', 12000),
(107, 'Backpack', 'Bags', 1800),
(108, 'USB Cable', 'Accessories', 500);

INSERT INTO orders VALUES
(1001, 1, '2026-09-01', 57500),
```

```

(1002,2,'2026-09-02',25000),
(1003,3,'2026-09-03',3300),
(1004,1,'2026-09-05',13800),
(1005,4,'2026-09-06',1800),
(1006,5,'2026-09-08',55000),
(1007,6,'2026-09-10',3300),
(1008,7,'2026-09-12',12500),
(1009,3,'2026-09-15',25000),
(1010,8,'2026-09-18',2300);

INSERT INTO order_items VALUES
(1,1001,101,1,55000),(2,1001,105,1,800),(3,1001,104,1,1500),
(4,1002,102,1,25000),(5,1003,103,1,2500),(6,1003,105,1,800),
(7,1004,106,1,12000),(8,1004,104,1,1500),(9,1004,105,1,800),
(10,1005,107,1,1800),(11,1006,101,1,55000),
(12,1007,103,1,2500),(13,1007,105,1,800),
(14,1008,106,1,12000),(15,1008,108,1,500),
(16,1009,102,1,25000),(17,1010,107,1,1800),(18,1010,105,1,500);

-- Core analysis
SELECT * FROM customers;
SELECT * FROM products;

SELECT product_name, category, price
FROM products
WHERE category = 'Electronics';

SELECT product_name, category, price
FROM products
ORDER BY price DESC;

SELECT category, COUNT(*) AS product_count
FROM products
GROUP BY category;

SELECT SUM(total_amount) AS total_revenue
FROM orders;

SELECT ROUND(AVG(total_amount),2) AS average_order_value
FROM orders;

SELECT o.order_id, c.customer_name, c.city,
       o.order_date, o.total_amount
FROM orders o
INNER JOIN customers c
       ON o.customer_id = c.customer_id;

SELECT c.customer_id, c.customer_name,
       o.order_id, o.total_amount
FROM customers c
LEFT JOIN orders o
       ON c.customer_id = o.customer_id;

SELECT c.customer_name,
       SUM(o.total_amount) AS total_revenue
FROM customers c
INNER JOIN orders o
       ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_revenue DESC;

SELECT p.product_name,
       SUM(oi.quantity * oi.unit_price) AS product_revenue
FROM products p
INNER JOIN order_items oi
       ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY product_revenue DESC;

```

```
SELECT ROUND(
    SUM(total_amount) * 1.0 /
    COUNT(DISTINCT customer_id), 2
) AS average_revenue_per_user
FROM orders;

SELECT order_id, customer_id, order_date, total_amount
FROM orders
WHERE total_amount > (
    SELECT AVG(total_amount)
    FROM orders
)
ORDER BY total_amount DESC;

SELECT customer_id, customer_name,
    COALESCE(email, 'No Email') AS email
FROM customers;

CREATE VIEW customer_revenue AS
SELECT c.customer_id, c.customer_name, c.city,
    SUM(o.total_amount) AS total_revenue
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name, c.city;

SELECT * FROM customer_revenue
ORDER BY total_revenue DESC;

CREATE INDEX idx_orders_customer_id
ON orders(customer_id);

CREATE INDEX idx_orders_order_date
ON orders(order_date);

CREATE INDEX idx_products_category
ON products(category);

EXPLAIN QUERY PLAN
SELECT *
FROM orders
WHERE customer_id = 1;
```